ARIEMEDI

精准导航　智引未来

瑞瞳 SDK 开发说明文档

Windows Python 版

版本号 V4.8.0

型号规格 RUITONG SDK

艾瑞迈迪医疗科技（北京）有限公司

目录

1. 简介 4
1.1 产品概述 4
1.2 环境要求 4
1.2.1 系统要求 4
1.2.2 开发环境与编译语言 4
2. SDK 基础接口说明 4
2.1 设备连接接口 4
2.1.1 autoDeviceScan 4
2.1.2 getScannedDevice 4
2.1.3 connectDevice 5
2.1.4 disconnectDevice 5
2.1.5 getDeviceInfo 5
2.2 器械设置接口 6
2.2.1 loadPassiveToolAROM 6
2.2.2 getTrackToolsNum 6
2.2.3 getTrackToolsName 6
2.2.4 generateAROM 7
2.2.5 getTrackToolsCalibration 7
2.3 数据传输接口 8
2.3.1 setTrackingDataTransmissionType 8
2.3.2 startTracking 8
2.3.3 stopTracking 9
2.3.4 startImaging 9
2.3.5 stopImaging 9
2.3.6 startMonitoring 9
2.3.7 stopMonitoring 10
2.3.8 getTrackingToolCurrentNum 10
2.3.9 trackingUpdate 10
2.3.10 getAllMarkers 11
2.3.11 getTrackingData 11
2.3.12 getToolMarkerData 11
2.3.13 getToolVirtualPointData 12
2.3.14 getUnMatchedMarkers 12
2.3.15 monitoringUpdate 12
2.3.16 getLeftImagingData 13
2.3.17 getRightImagingData 13
2.3.18 getIFImageSize 13
2.3.19 getMonitorImagingData 14
2.3.20 getMonitorImageSize 14
2.3.21 setMonitorImageResolution 14
2.4 状态监测接口 15
2.4.1 getConnectionStatus 15

2.4.2 getSystemAlert 15
2.4.3 getGravityVector 16
2.5 常用工具接口 16
2.5.1 setAreaDisplay 16
2.5.2 setAreaHidden 16
2.5.3 beep 17
2.5.4 laser 17
2.5.5 laserOn 17
2.5.6 getNetAdaptor 18
2.5.7 saveLog 18
2.5.8 setReferenceTool 18
2.5.9 trackingDataProjection 19
2.5.10 trackingDataProjection2 19
3. SDK 重建接口说明 19
3.1 setMaterial 19
3.2 reconstruction 20
3.3 getPointCloud 20
3.4 getPointNum 21
3.5 getTextureImageUChar 21
3.6 getDepthImage 21
3.7 getDepthImageSize 21
3.8 project2DImagePosition 22
3.9 savePointCloud 22
4. SDK 重要数据结构 22
4.1 器械标定数据结构 ToolCalibration 22
4.2 点（云）数据结构 PointInf 23
4.3 跟踪信息数据结构 TrackingData 23
4.4 标志物数据结构 MarkerPosition 24
4.5 重力向量结构 DirVector 24
4.6 网卡信息结构 NetworkAdaptorInfo 24
4.7 设备信息结构 RTDeviceInfo 24
5. Demo 使用指南 24
5.1 连接准备 24
5.2 文件目录 25
5.3 工程配置 25
5.4 调用流程 25
5.4.1 基础示例调用流程 25
5.4.2 跨平台跟踪数据发送示例调用流程 25
5.4.3 点云重建示例调用流程 26
6 常见问题 26

1. 简介

1.1 产品概述

适用于瑞瞳系列产品（瑞瞳 SE、瑞瞳 LITE、瑞瞳 MAX、瑞瞳 MAXV、瑞瞳 PRO）的 SDK 工具包，包含瑞瞳的连接与初始化、标志物定位、器械识别跟踪、表面点云重建等功能，用户可根据业务需求结合 SDK 灵活的进行应用层开发。

1.2 环境要求

1.2.1 系统要求

Windows 10/11 系统

1.2.2 开发环境与编译语言

PyCharm, Python

2. SDK 基础接口说明

2.1 设备连接接口

功能包括：连接、断开、获取设备信息。

2.1.1 autoDeviceScan

原型

autoDeviceScan(num)

功能描述

自动扫描局部网内可连接的设备数量。

参数

num：ctypes.byref(ctypes.c_int())类型，存放扫描到的局部网内可连接设备数量。

返回值

无。

2.1.2 getScannedDevice

原型

getScannedDevice(i, hostname, ip)

功能描述

获取自动扫描到的指定编号的局部网内可连接设备 hostname 和 IP 地址。

参数

① i：数字类型，整数，指定设备编号，从 0 开始；

② hostname：StringBuilder 类型，用于存放指定设备的 hostname；

③ ip：StringBuilder 类型，用于存放指定设备的 IP 地址。

返回值

无。

2.1.3 connectDevice

原型

connectDevice(hostname/IP)

功能描述

连接与初始化设备。

参数

hostname/IP：字符串类型，设备对应的 hostname 或 IP。

返回值

-1：连接失败；

0：连接成功。

2.1.4 disconnectDevice

原型

disconnectDevice()

功能描述

断开与设备的连接。

参数

无。

返回值

无。

2.1.5 getDeviceInfo

原型

getDeviceInfo()

功能描述

获取已连接的设备信息。

参数

无。

返回值

已连接的设备信息：RTDeviceInfo 类型。该类型信息详见 4.7 节。

2.2 器械设置接口

功能包括：导入\生成\保存器械标定文件、获取器械数量。

2.2.1 loadPassiveToolAROM

原型

loadPassiveToolAROM(directory)

功能描述

从指定文件夹目录读取所有器械标定文件（*.arom）。

参数

directory：器械标定文件所在文件夹。

返回值

无。

2.2.2 getTrackToolsNum

原型

getTrackToolsNum()

功能描述

获取待跟踪所有器械数量。

参数

无。

返回值

待跟踪器械数量：其与读入的器械标定文件（*.arom）数量一致。

2.2.3 getTrackToolsName

原型

getTrackToolsName(index, str, strLen)

功能描述

获取指定器械名称。

参数

① index：指定器械的编号，其范围为 0~(所有跟踪器械数量-1)；

② str：StringBuilder，用于存放指定器械的名称；

③ strLen：指定器械名称的长度。

返回值

无。

2.2.4 generateAROM

原型

generateAROM(directory, tool, markerPlaneNum, markerPlaneDir, markers, points, radius)

功能描述

根据用户输入的参数，生成器械标定文件（*.arom）。**注意**：该接口函数无法替代器械尖端标定过程，若要确定器械尖端坐标，请使用标定软件或调用 pivotTipCalibration 函数。

参数

① directory：字符串类型，生成路径；

② tool：ToolInf 类型，器械信息结构体；

③ markerPlaneNum：ctypes.byref(ctypes.c_int*p)类型，器械各平面内标志物数量，其中 p 为平面数量；

④ markerPlaneDir：ctypes.byref(DirVector*p)类型，器械各平面法向量，其中 p 为平面数量，DirVector 类型信息详见 4.5 节。；

⑤ markers：ctypes.byref(MarkerPosition*m)类型，器械所有平面标志物信息，各平面内标志物依次排列，其中 m 为器械每个平面标志物数量之和；

⑥ points：ctypes.byref(MarkerPosition*n)类型，器械所有虚拟点信息，其中 n 为虚拟点数量；

⑦ radius：double 类型，标志物半径，单位：毫米。

返回值

生成结果：生成器械标定文件（*.arom）成功或失败信息，其中 0 为成功；1 为生成失败，最少匹配标志物数量过少；2 为生成失败，最少匹配标志物数量过多；3 为生成失败，平面数量过少；4 为生成失败，平面数量过多；5 为生成失败，平面数量不匹配；6 为生成失败，平面内标志物数量过少；7 为生成失败，平面内标志物数量过多；8 为生成失败，器械标志物数量过多；9 为生成失败，标志物间距过小；10 为生成失败，文件名包含中文与符号；11 为生成失败，文件存储路径不存在；12 为生成失败，文件类型错误。

2.2.5 getTrackToolsCalibration

原型

getTrackToolsCalibration(index)

功能描述

获取指定器械标定信息。

参数

index：数字类型，整数，指定器械的编号，其范围为 0~(所有跟踪器械数量-1)。

返回值

指定器械标定信息：ToolCalibration 类型。该类型信息详见 4.1 节。

2.3 数据传输接口

功能包括：刷新图像和标志物跟踪数据、获取图像与器械跟踪数据。

2.3.1 setTrackingDataTransmissionType

原型

setTrackingDataTransmissionType(type)

功能描述

设置传输跟踪数据的标志物类型。

参数

type：数字类型，表示标志物类型，其中 1 为只传输被动反光标志物跟踪数据，2 为只传输主动发光标志物跟踪数据，3 为混合不区分地发送被动反光与主动发光标志物跟踪数据，4 为间隔区分地发送被动反光与主动发光标志物跟踪数据。注意：若不设置，瑞瞳设备默认只传输被动反光标志物跟踪数据，且在混合传输模式下无法区分主动反光与被动发光标志物跟踪数据。

返回值

无。

2.3.2 startTracking

原型

startTracking()

功能描述

开始跟踪器械。

参数

无。

返回值

无。

2.3.3 stopTracking

原型

stopTracking()

功能描述

停止跟踪器械。

参数

无。

返回值

无。

2.3.4 startImaging

原型

startImaging()

功能描述

开始传输双目红外图像。

参数

无。

返回值

无。

2.3.5 stopImaging

原型

stopImaging()

功能描述

停止传输双目红外图像。

参数

无。

返回值

无。

2.3.6 startMonitoring

原型

startMonitoring(r)

功能描述

开始传输可见光图像数据，图像默认为高分辨率尺寸 1920*1200。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号。

参数

r：bool 类型，true 为高分辨率（1920*1200），false 为低分辨率（1280*720）。

返回值

无。

2.3.7 stopMonitoring

原型

stopMonitoring()

功能描述

停止传输可见光图像数据。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号。

参数

无。

返回值

无。

2.3.8 getTrackingToolCurrentNum

原型

getTrackingToolCurrentNum()

功能描述

获取当前跟踪的所有器械数量。

参数

无。

返回值

器械数量：当前跟踪的所有器械数量。

2.3.9 trackingUpdate

原型

trackingUpdate()

功能描述

刷新所有标志物跟踪数据以及双目图像数据。

参数

无。

返回值

无。

2.3.10 getAllMarkers

原型

getAllMarkers(num)

功能描述

获取当前视场中所有标志物的跟踪数据。

参数

num：ctypes.POINTER(ctypes.c_int)类型，为 C 风格 int 型指针，用于储存当前视场中所有标志物的数量。

返回值

所有标志物的跟踪数据：ctypes.POINTER(MarkerPosition)类型，为 C 风格 MarkerPosition 型指针，用于储存当前视场中所有标志物的跟踪数据。该类型信息详见 4.4 节。

2.3.11 getTrackingData

原型

getTrackingData()

功能描述

获取所有器械的跟踪数据。

参数

无。

返回值

所有器械的跟踪数据：ctypes.POINTER(TrackingData)类型，为 C 风格 TrackingData 型指针，用于储存当前视场中所有器械的跟踪数据，包括跟踪状态、器械位姿、跟踪误差等。该类型信息详见 4.3 节。

2.3.12 getToolMarkerData

原型

getToolMarkerData(index)

功能描述

获取指定器械上的标志物跟踪数据。

参数

index：器械编号，其范围为 0~(所有跟踪器械数量-1)。

返回值

器械上的标志物跟踪数据：ctypes.POINTER(MarkerPosition)类型，为 C 风格 MarkerPosition 型指针，用于储存当前指定器械上的标志物跟踪数据。该类型信息详见 4.4 节。

2.3.13 getToolVirtualPointData

原型

getToolVirtualPointData(index)

功能描述

获取指定器械上的虚拟点跟踪数据。

参数

index：器械编号，其范围为 0~(所有跟踪器械数量-1)。

返回值

器械的虚拟点跟踪数据：ctypes.POINTER(MarkerPosition)类型，为 C 风格 MarkerPosition 型指针，用于储存当前指定器械虚拟店跟踪数据。该类型信息详见 4.4 节。

2.3.14 getUnMatchedMarkers

原型

getUnMatchedMarkers(num)

功能描述

获取当前视场中所有游离的未匹配标志物跟踪数据。

参数

num：ctypes.POINTER(ctypes.c_int)类型，为 C 风格 int 型指针，用于储存当前视场中所有游离的未匹配标志物数量。

返回值

游离的未匹配标志物跟踪数据：ctypes.POINTER(MarkerPosition)类型，为 C 风格 MarkerPosition 型指针，用于储存当前视场中所有游离的未匹配标志物跟踪数据。该类型信息详见 4.4 节。

2.3.15 monitoringUpdate

原型

monitoringUpdate()

功能描述

刷新可见光图像数据。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号。

参数

无。

返回值

无。

2.3.16 getLeftImagingData

原型

getLeftImagingData()

功能描述

获取当前左目相机图像，图像尺寸为 1280*800 像素（SE、LITE、MAX 与 MAXV 型号）或 1920*1200 像素（PRO 型号）。

参数

无。

返回值

当前左目相机图像数据：ctypes.POINTER(ctypes.c_byte)类型。

2.3.17 getRightImagingData

原型

getRightImagingData()

功能描述

获取当前右目相机图像，图像尺寸为 1280*800 像素（SE、LITE、MAX 与 MAXV 型号）或 1920*1200 像素（PRO 型号）。

参数

无。

返回值

当前右目相机图像数据：ctypes.POINTER(ctypes.c_byte)类型。

2.3.18 getIFImageSize

原型

getIFImageSize(size)

功能描述

获取双目相机图像尺寸。

参数

size：ctypes.POINTER(ctypes.c_int)类型，返回双目相机图像长宽。

返回值

无。

2.3.19 getMonitorImagingData

原型

getMonitorImagingData()

功能描述

获取当前可见光相机图像，高分辨率图像尺寸为 1920*1200 像素，低分辨率图像尺寸为 1280*720 像素。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号。

参数

无。

返回值

当前可见光相机图像数据：ctypes.POINTER(ctypes.c_ubyte)类型。

2.3.20 getMonitorImageSize

原型

getMonitorImageSize(size)

功能描述

获取可见光相机图像尺寸。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号。

参数

size：ctypes.POINTER(ctypes.c_int)类型，返回可见光相机图像长宽。

返回值

无。

2.3.21 setMonitorImageResolution

原型

setMonitorImageResolution(r)

功能描述

设置可见光相机图像分辨率。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号，若不设置默认为高分辨率。

参数

r：bool 类型，false 为低分辨率（1280*720），true 为高分辨率（1920*1200）。

返回值

无。

2.4 状态监测接口

功能包括：获取设备连接状态、系统警告、重力向量。

2.4.1 getConnectionStatus

原型

getConnectionStatus()

功能描述

获取当前设备连接状态。

参数

无。

返回值

0：设备已连接；

1：设备已断开；

2：设备断开失败；

3：端口冲突，设备连接失败；

4：设备信息网络解析错误，设备连接失败；

5：连接初始化错误，设备连接失败；

6：设备中断；

7：无效连接地址，设备连接失败；

8：局域网内未检测到设备，设备连接失败；

9：未知错误，设备连接失败。

2.4.2 getSystemAlert

原型

getSystemAlert()

功能描述

获取当前设备警告信息。

参数

无。

返回值

0：设备正常；

1：设备发生过大震动，请返厂维修；

2：设备温度过高；

3：设备温度过低。

2.4.3 getGravityVector

原型

getGravityVector()

功能描述

获取当前设备重力向量。

参数

无。

返回值

重力向量：DirVector 类型，当前设备重力单位向量。该类型信息详见 4.5 节。

2.5 常用工具接口

功能包括：标定器械跟踪数据投影、在双目红外图像上绘制重建的矩形区域、设备蜂鸣、开启定位激光。

2.5.1 setAreaDisplay

原型

setAreaDisplay(width, height, leftX, leftY, rightX, rightY)

功能描述

在双目图像上绘制矩形框。

参数

① width：数字类型，整数，表示绘制矩形区域长度（单位：像素）；

② height：数字类型，整数，表示绘制矩形区域宽度（单位：像素）；

③ leftX：数字类型，整数，表示绘制矩形区域在左目图像 x 方向上起始坐标；

④ leftY：数字类型，整数，表示绘制矩形区域在左目图像 y 方向上起始坐标；

⑤ rightX：数字类型，整数，表示绘制矩形区域在右目图像 x 方向上起始坐标；

⑥ rightY：数字类型，整数，表示绘制矩形区域在右目图像 y 方向上起始坐标。

返回值

无。

2.5.2 setAreaHidden

原型

setAreaHidden()

功能描述

隐藏在双目图像上绘制的矩形框。

参数

无。

返回值

无。

2.5.3 beep

原型

beep(delay)

功能描述

开始蜂鸣，且将在设定延迟时间后关闭。

参数

delay：数字类型，整数，延迟时间，单位：毫秒。

返回值

无。

2.5.4 laser

原型

laser(s)

功能描述

开启/关闭定位激光。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号。

参数

s：bool 类型，开启（true），关闭（false）。

返回值

无。

2.5.5 laserOn

原型

laserOn(delay)

功能描述

打开定位激光，且激光将在设定延迟时间后关闭。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号。

参数

delay：数字类型，整数，表示延迟时间，单位：毫秒。

返回值

无。

2.5.6 getNetAdaptor

原型

getNetAdaptor()

功能描述

获取与设备连接的网卡信息。

参数

无。

返回值

网卡信息：NetworkAdaptorInfo 类型，包括网卡名称、连接方式、网速（单位：Mb/s）、连接状态。该类型信息详见 4.6 节。

2.5.7 saveLog

原型

saveLog(path)

功能描述

保存日志信息。

参数

path：字符串类型，保存路径及文件名，例./record.log。

返回值

保存结果：bool 类型，成功 True，失败 False，其中失败原因可能为保存路径不存在。

2.5.8 setReferenceTool

原型

setReferenceTool(name)

功能描述

设置参考器械，函数 getTrackingData()返回的第一个数据为参考工具的跟踪数据。

参数

name：字符串类型，参考器械名称。

返回值

无。

2.5.9 trackingDataProjection

原型

trackingDataProjection(ref, src)

功能描述

将源器械跟踪数据投影至参考器械坐标系。

参数

① ref：TrackingData 类型，参考器械跟踪数据

② src：TrackingData 类型，源器械跟踪器械

返回值

投影后源器械跟踪数据：ctypes.POINTER(TrackingData)类型，为 C 风格 TrackingData 型指针，用于储存投影结果。该类型信息详见 4.3 节。

2.5.10 trackingDataProjection2

原型

trackingDataProjection2(ref, src, markers/points, flag)

功能描述

将源器械标志物/虚拟点跟踪数据投影至参考器械坐标系。

参数

① ref：TrackingData 类型，参考器械跟踪数据

② src：TrackingData 类型，源器械跟踪数据

③ markers：ctypes.POINTER(MarkerPosition)类型，源器械标志物/虚拟点跟踪数据

④ flag：ctypes.c_bool 类型，标志物/虚拟点标识，true-标志物，false-虚拟点

返回值

投影后源器械标志物/虚拟点跟踪数据：ctypes.POINTER(MarkerPosition)类型，为 C 风格 TrackingData 型指针，用于储存投影结果。该类型信息详见 4.3 节。

3. SDK 重建接口说明

功能包括：在重建、保存点云。**注意**：该接口函数仅适用于 PRO 型号。

3.1 setMaterial

原型

setMaterial(material)

功能描述

设置重建物体的材质。注意：当前仅支持硅胶（0）与石膏材质（1），默认为硅胶材质。

参数

material：数字类型，整数，表示材质编号。

返回值

无。

3.2 reconstruction

原型

reconstruction(width, height, leftX, leftY, rightX, rightY)

功能描述

重建指定范围内的点云。

参数

① width：数字类型，整数，表示重建区域长度（单位：像素）；

② height：数字类型，整数，表示重建区域宽度（单位：像素）；

③ leftX：数字类型，整数，表示重建区域在左目图像 x 方向上的起始坐标；

④ leftY：数字类型，整数，表示重建区域在左目图像 y 方向上的起始坐标；

⑤ rightX：数字类型，整数，表示重建区域在右目图像 x 方向上的起始坐标；

⑥ rightY：数字类型，整数，表示重建区域在右目图像 y 方向上的起始坐标。

返回值

无。

3.3 getPointCloud

原型

getPointCloud(index)

功能描述

获取点云中指定编号点的信息。

参数

index：数字类型，整数，点编号，从 0 开始。

返回值

点云中指定编号点的信息：PointInf 类型，该类型信息详见 4.2 节。

3.4 getPointNum

原型

getPointNum()

功能描述

获取点云中点数量。

参数

无。

返回值

点数量：数字类型，整数。

3.5 getTextureImageUChar

原型

getTextureImageUChar()

功能描述

获取点云纹理的可见光图像，其尺寸为 1920*1200。

参数

无。

返回值

纹理图像：ctypes.POINTER(ctypes.c_ubyte)类型，点云纹理的可见光图像。

3.6 getDepthImage

原型

getDepthImage(depthImage)

功能描述

获取点云重建的深度图。

参数

深度图：ctypes.POINTER(ctypes.c_float)类型，存放点云重建的深度图。

返回值

无。

3.7 getDepthImageSize

原型

getDepthImageSize(size)

功能描述

获取点云重建的深度图尺寸。

参数

size：ctypes.POINTER(ctypes.c_int)类型，存放返回深度图长宽。

返回值

无。

3.8 project2DImagePosition

原型

project2DImagePosition(x, y)

功能描述

获取可见光图像坐标对应的点（云）编号，其中图像坐标原点为左上角。

参数

① x：数字类型，整数，表示可见光图像点坐标 x（单位：像素）；

② y：数字类型，整数，表示可见光图像点坐标 y（单位：像素）。

返回值

点（云）编号：数字类型，整数，若输入的可见光图像坐标无对应的点（云），则返回-1。

3.9 savePointCloud

原型

savePointCloud(path, texture)

功能描述

将点云保存至指定路径。

参数

① path：字符串类型，保存路径；

② texture：bool 类型，是否写入纹理。

返回值

保存结果：bool 类型，True 保存成功，False 保存失败。

4. SDK 重要数据结构

4.1 器械标定数据结构 ToolCalibration

```
class ToolCalibration(ctypes.Structure):
    _fields_ = [("name", ctypes.c_char_p),\        #器械名称
    ("markerType", ctypes.c_bool),\                #标志物类型，False-被动，True-主动
```

```
("minNumMarker", ctypes.c_int),\          #标志物最小匹配数量
("planeNum", ctypes.c_int),\              #器械平面数量
("pointNum", ctypes.c_int),\              #器械虚拟点数量
("dirx", ctypes.c_double),\               #虚拟方向向量 x 分量
("diry", ctypes.c_double),\               #虚拟方向向量 y 分量
("dirz", ctypes.c_double),\               #虚拟方向向量 z 分量
("pinx", ctypes.c_double),\               #尖端坐标 x
("piny", ctypes.c_double),\               #尖端坐标 y
("pinz", ctypes.c_double),\               #尖端坐标 z
("calbError", ctypes.c_double),\          #标定误差
("maxFRE", ctypes.c_double),\             #最大匹配误差
("maxAngle", ctypes.c_double),            #最大跟踪角度
("structureType", ctypes.c_int),\         #器械结构，0 单平面，1 多平面，2 非平面
("algorithmType", ctypes.c_int)]          #匹配方式，0 平衡，1 精度优先，2 速度优先
```

4.2 点（云）数据结构 PointInf

```
class PointInf(ctypes.Structure):
    _fields_ = [("x", ctypes.c_float),\       #点坐标 x，单位：毫米
    ("y", ctypes.c_float),\                   #点坐标 y，单位：毫米
    ("z", ctypes.c_float),\                   #点坐标 z，单位：毫米
    ("r", ctypes.c_float),\                   #点纹理颜色 r
    ("g", ctypes.c_float),\                   #点纹理颜色 g
    ("b", ctypes.c_float)]                    #点纹理颜色 b
```

4.3 跟踪信息数据结构 TrackingData

```
class TrackingData(ctypes.Structure):
    _fields_ = [("matchStatus", ctypes.c_int),\    #匹配状态：1（跟踪），0（丢失）
    ("matchedPlane", ctypes.c_int),\               #匹配平面编号，从 1 开始
    ("markerNum", ctypes.c_int),\                  #匹配平面上标志物数量
    ("pointNum", ctypes.c_int),\                   #虚拟点数量
    ("tx", ctypes.c_double),\                      #尖端坐标 x，单位：毫米
    ("ty", ctypes.c_double),\                      #尖端坐标 y，单位：毫米
    ("tz", ctypes.c_double),\                      #尖端坐标 z，单位：毫米
    ("ox", ctypes.c_double),\                      #偏移量 x，单位：毫米
    ("oy", ctypes.c_double),\                      #偏移量 y，单位：毫米
    ("oz", ctypes.c_double),\                      #偏移量 z，单位：毫米
    ("qw", ctypes.c_double),\                      #姿态四元数 w
    ("qx", ctypes.c_double),\                      #姿态四元数 x
    ("qy", ctypes.c_double),\                      #姿态四元数 y
    ("qz", ctypes.c_double),\                      #姿态四元数 z
    ("rxx", ctypes.c_double),\                     #旋转矩阵(0,0)
    ("ryx", ctypes.c_double),\                     #旋转矩阵(0,1)
    ("rzx", ctypes.c_double),\                     #旋转矩阵(0,2)
    ("rxy", ctypes.c_double),\                     #旋转矩阵(1,0)
    ("ryy", ctypes.c_double),\                     #旋转矩阵(1,1)
```

```
        ("rzy", ctypes.c_double),\                  #旋转矩阵(1,2)
        ("rxz", ctypes.c_double),\                  #旋转矩阵(2,0)
        ("ryz", ctypes.c_double),\                  #旋转矩阵(2,1)
        ("rzz", ctypes.c_double),\                  #旋转矩阵(2,2)
        ("dx", ctypes.c_double),\                   #虚拟方向 x
        ("dy", ctypes.c_double),\                   #虚拟方向 y
        ("dz", ctypes.c_double),\                   #虚拟方向 z
        ("error", ctypes.c_double)]                 #匹配误差
```

4.4 标志物数据结构 MarkerPosition

```
class MarkerPosition(ctypes.Structure):
    _fields_ = [("x", ctypes.c_double),\            #标志物坐标 x，单位：毫米
    ("y", ctypes.c_double),\                        #标志物坐标 y，单位：毫米
    ("z", ctypes.c_double),\                        #标志物坐标 z，单位：毫米
    ("type", ctypes.c_int),\                        #标志物类型：0-未知，1-被动，2-主动
    ("RF", ctypes.c_bool)]                          #标志物识别结果，true-真实，false-疑似
```

4.5 重力向量结构 DirVector

```
class DirVector(ctypes.Structure):
    _fields_ = [("vx", ctypes.c_double),\           #设备重力向量 x
    ("vy", ctypes.c_double),\                       #设备重力向量 y
    ("vz", ctypes.c_double)]                        #设备重力向量 z
```

4.6 网卡信息结构 NetworkAdaptorInfo

```
class NetworkAdaptorInfo(ctypes.Structure):
    _fields_ = [("name", ctypes.c_char_p),\         #名称
    ("connectionType ", ctypes.c_char_p),\          #连接类型
    ("speed", ctypes.c_int),\                       #网速
    ("linked", ctypes.c_bool)]                      #连接状态
```

4.7 设备信息结构 RTDeviceInfo

```
class RTDeviceInfo(ctypes.Structure):
    _fields_ = [("hostname", ctypes.c_char_p),\     #hostname
    ("IP", ctypes.c_char_p),\                       #IP 地址
    ("MAC", ctypes.c_ char_p),\                     #MAC 地址
    ("deviceType", ctypes.c_int),\                  #设备型号
    ("firmwareVersionL", ctypes.c_int),\            #固件版本号低位
    ("firmwareVersionM", ctypes.c_int),\            #固件版本号中位
    ("firmwareVersionH", ctypes.c_int)]             #固件版本号高位
```

5. Demo 使用指南

SDK 的 Windows Python 版本 Demo 使用指南以 PyCharm 平台为例，测试该 Demo 的 Python 版本为 3.6，PyCharm 版本为 Community-2024.1.3。

5.1 连接准备

查看接入瑞瞳设备背面标记的 hostname，例如 RT-SE000001.local。

5.2 文件目录

ARMDCSDLLWarpper.dll、ARMDCSDLLWarpperPro.dll 与 ARMDstereotracking.dll、ARMDstereoreconstruction.dll 为 SDK 库文件，opencv_world452.dll、opencv_img_hash452.dll 为第三方 Opencv 库文件用于瑞瞳 PRO 设备的重建功能，main.py、mainPro.py、mainIGT.py 为调用 SDK 的 Python 示例代码，Tool 文件夹包含器械的标定文件（*.arom）。

5.3 工程配置

将*.py 与库文件、Tool 文件夹放置在同级目录下。**注意**：若用户仅使用跟踪功能，只需要 ARMDCSDLLWarpper.dll 与 ARMDstereotracking.dll 库文件；若用户使用跟踪+重建功能，需要 ARMDCSDLLWarpperPro.dll 与 ARMDstereotracking.dll、ARMDstereoreconstruction.dll 以及 Opencv 库文件。

5.4 调用流程

main.py 为包含设备连接、状态监测、标志物定位、器械跟踪的基础示例，mainIGT.py 为包含基础功能与以 IGT 协议发送跟踪数据的跨平台跟踪数据发送示例，mainPro.py 为包含基础功能与重建功能的点云重建示例。

5.4.1 基础示例调用流程

Step1：读入 ARMDCSDLLWarpper.dll 库

读入 SDK 的库文件。

Step2：连接设备

连接并初始化设备。

Step3：读取标定文件

在指定路径下，读取所有器械标定文件（*.arom）。

Step4：开启子线程，更新视场中器械状态

在子线程中，循环更新当前视场中所有标志物与器械跟踪数据，包括标志位坐标以及器械的配准误差、姿态四元数和尖端点坐标。

5.4.2 跨平台跟踪数据发送示例调用流程

Step1：读入 ARMDCSDLLWarpper.dll 库

读入 SDK 的库文件。

Step2：连接设备

连接并初始化设备。

Step3：建立 IGT 通讯

建立 IGT 服务端，并等待客户端（如 3DSlicer）连接。

Step4：读取标定文件

在指定路径下，读取所有器械标定文件（*.arom）。

Step5：开启子线程，更新视场中器械状态

在子线程中，循环更新当前视场中所有标志物与器械跟踪数据，包括标志位坐标以及器械的配准误差、姿态四元数和尖端点坐标。

Step5：通过 IGT 协议，将器械跟踪数据发送至客户端 •

将器械跟踪数据转换为数组，并通过 IGT 协议发送至连接的客户端（如 3DSlicer）。

5.4.3 点云重建示例调用流程

Step1：读入 ARMDCSDLLWarpperPro.dll 库

读入 SDK 的库文件。

Step2：连接设备

连接并初始化设备。

Step3：读取标定文件

在指定路径下，读取所有器械标定文件（*.arom）。

Step4：开启子线程，更新视场中器械状态

在子线程中，循环更新当前视场中所有标志物与器械跟踪数据，包括标志位坐标以及器械的配准误差、姿态四元数和尖端点坐标。

Step5：重建

在主线程启动重建功能，重建完成后，可查看纹理图、深度图以及保存重建点云。

6 常见问题

Q：启动系统时显示连接失败。

A：检查连接时输入的 hostname 是否与设备背面标记的一致。

Q：手动连接设备成功，但自动连接设备失败。

A：请关闭所有防火墙后，重新自动连接。